

Spiking neural networks for streaming qubit readout

Barry M. Dillon¹, Aqib Javed¹, Jim Harkin¹,
 Patryk Dąbkowski^{2,3,4,5}, Benjamin Lienhard^{2,3,4}

¹ ISRC, Ulster University, Derry, BT48 7JL, Northern Ireland

² Technical University of Munich, Department of Physics, 85748 Garching, Germany

³ Walther-Meißner-Institut, Bayerische Akademie der Wissenschaften, 85748 Garching, Germany

⁴ Munich Center for Quantum Science and Technology, 80799 Munich, Germany

⁵ Zurich Instruments, 8005 Zürich, Switzerland

`b.dillon@ulster.ac.uk`

Abstract

Fast and accurate qubit-state assignment is essential for feedback, calibration, and error correction in quantum processors. In superconducting platforms, frequency-multiplexed readout makes this task intrinsically multivariate as measured traces can encode crosstalk, qubit-state relaxation events, and other transient nonidealities that are not fully captured by conventional matched filtering. Here, we introduce spiking neural network discriminators for superconducting qubit readout. By processing the measurement window in successive time chunks, the networks exploit temporal structure and update classification scores as data arrive, rather than waiting until the end of the readout window. The spiking networks outperform matched-filter discrimination and approach the accuracy of a full-trace artificial neural network. Beyond reaching the performance of artificial neural networks, the key advantage of SNNs is that they provide a streaming, time-resolved estimate of the qubit state that evolves as the readout signal is acquired. Using quantisation-aware training and `hls4ml` synthesis, we further demonstrate that each FPGA inference update can be completed before the next readout chunk arrives. These results establish spiking neural networks as a promising route to low-latency, real-time qubit readout on FPGA hardware, with broader implications for time-critical quantum-control and scientific-inference applications.

Keywords: superconducting-qubit readout; spiking neural networks; FPGA inference; quantisation-aware training; `hls4ml`

1 Introduction

Scaling quantum computation from a few tens to tens of thousands of qubits promises new capabilities in science and technology, from more accurate simulations of molecules and materials to advances in chemistry, drug discovery, and complex optimisation [1]. Realising this promise requires suppressing and managing errors through quantum error correction

(QEC) and error mitigation, both of which rely on repeated, high-fidelity access to the processor [2–4]. Readout, the measurement step that converts quantum information into a classical record, is therefore not merely a final diagnostic operation, but a central component of scalable quantum computation.

For many quantum-computing platforms, readout remains among the most error-prone and time-consuming operations. This is particularly consequential in feedback-based protocols, calibration routines, and QEC cycles, where decisions must be made rapidly enough to act before encoded information changes [5]. Improving readout, therefore, requires addressing two coupled objectives: increasing state-assignment accuracy while reducing the latency between measurement acquisition and classification.

Superconducting qubits are among the leading platforms for realising useful quantum processors. They combine fast gate operations with a mature ecosystem for microwave control and measurement, which naturally supports parallel, frequency-multiplexed readout [6]. These features make superconducting architectures well-suited to QEC codes and scalable quantum hardware [7]. At the same time, superconducting devices remain vulnerable to hardware imperfections and environmental noise. In multiplexed readout, the measured signals can contain crosstalk, relaxation events, and transient nonidealities, making qubit-state assignment an intrinsically time-dependent and multivariate inference problem [8].

A common approach to qubit-state assignment in superconducting circuits is matched filtering (MF) [9]. In this method, readout traces are acquired for qubits prepared in known initial states and used to construct state-dependent templates. A new measurement record is then classified according to its overlap with these templates. This procedure typically requires acquiring the full trace before computing the MF response.

MF is simple and effective, but it has important limitations. Because it compresses the measurement record into a fixed overlap with a predetermined template, it is not generally optimised to detect rare or time-localised events that may occur at different points during the readout window, such as relaxation, transient crosstalk, or other nonideal dynamics [8]. Moreover, depending on the preprocessing required for the readout traces, MF evaluation can become computationally costly in latency-constrained settings.

To improve both readout accuracy and latency, machine-learning methods have been developed for superconducting-qubit-state assignment [8]. Artificial neural networks (ANNs), for example, can be trained directly on measurement records from qubits prepared in known states, learning a nonlinear map from the readout trace to the corresponding state assignment. More broadly, the literature on machine-learning-assisted readout includes Bayesian approaches [10], reservoir computing [11], and hybrid methods that combine neural networks with high-level MF-based observables [12]. We discuss this body of work in more detail in Sec. 2.1.

ANNs can improve both readout accuracy and inference latency when deployed on FPGA hardware [13]. In conventional implementations, however, the network is evaluated only after the full readout pulse has been acquired and preprocessed. The total readout latency is therefore the sum of the pulse duration and the network-inference latency. As FPGA-implemented neural networks can now achieve inference times on the order of 50 ns, the dominant latency bottleneck is no longer the network evaluation itself, but the need to wait for the complete readout record.

In this work, we propose using Spiking Neural Networks (SNNs) synthesised for FPGA

hardware to achieve high-accuracy, low-latency readout. The SNN discriminators process the readout signal sequentially as it arrives, rather than waiting until the readout has finished. This allows the classification process to start earlier, making better use of the available resources in a severely resource-limited setting. It also opens the possibility that time-dependent features could contribute to the classification before the full measurement window has elapsed. The idea is depicted schematically in Fig. 1. This approach provides a natural route to low-latency inference while retaining sensitivity to transient structures that conventional readout methods lack. Our results show that spiking neural networks can improve superconducting-qubit state assignment and provide a hardware-compatible path toward real-time readout for feedback, calibration, and quantum-error-correction workflows.

SNNs are well-suited to this task because they are both inherently temporal and are well-suited to quantisation and deployment in resource-limited settings. SNNs replace the continuous, stateless activations of ANNs with event-like spikes and an internal dynamical state that evolves over time [14, 15]. Here, we exploit this temporal memory to process successive chunks of the readout trace and update the qubit-state assignment throughout the acquisition. Recent advances in SNN synthesis for FPGA hardware using `hls4ml` [16] also enable realistic estimates of latency and resource requirements for deployment.

Together, these results establish streaming SNN-based discrimination as a neuromorphic route to low-latency qubit-state readout. By updating state assignments during acquisition, the method reduces the dependence of decision latency on the full readout-pulse duration while retaining sensitivity to transient features such as relaxation and crosstalk. Its combination of accuracy, low latency, and FPGA-compatible implementation addresses a key requirement for feedback, calibration, and QEC in superconducting quantum processors. More broadly, this work highlights neuromorphic architectures as real-time inference engines for time-critical quantum technologies.

2 Superconducting Qubits

Superconducting qubits encode quantum information in the lowest levels of weakly anharmonic Josephson-junction circuits and are typically measured dispersively through qubit-state-dependent shifts of microwave resonators. In the multiplexed-readout dataset introduced by Lienhard et al. [8], the experimental platform was a five-qubit transmon processor in which each qubit with transition frequencies between 4.3 GHz to 5.2 GHz and lifetimes ranging from 7 μ s to 40 μ s was coupled to an individual readout resonator in the range of 7 GHz to 7.25 GHz. All five resonators were coupled to a common feedline for frequency-multiplexed transmission measurement. This architecture provides a compact but realistic benchmark for multiqubit readout: it combines simultaneous dispersive measurement, shared analogue signal paths, resonator-frequency crowding, amplifier noise, non-Gaussian IQ distributions, inter-channel correlations, and readout crosstalk.

The dataset consists of labelled single-shot measurements acquired after preparing the device in the full set of $2^5 = 32$ computational-basis states. For each shot, the prepared basis state serves as the class label, and a frequency-multiplexed microwave readout pulse containing tones near the five readout-resonator frequencies is applied. The reflected signal is amplified, downconverted, digitised, and stored as time-resolved in-phase and quadra-

Quantum-computer readout chain

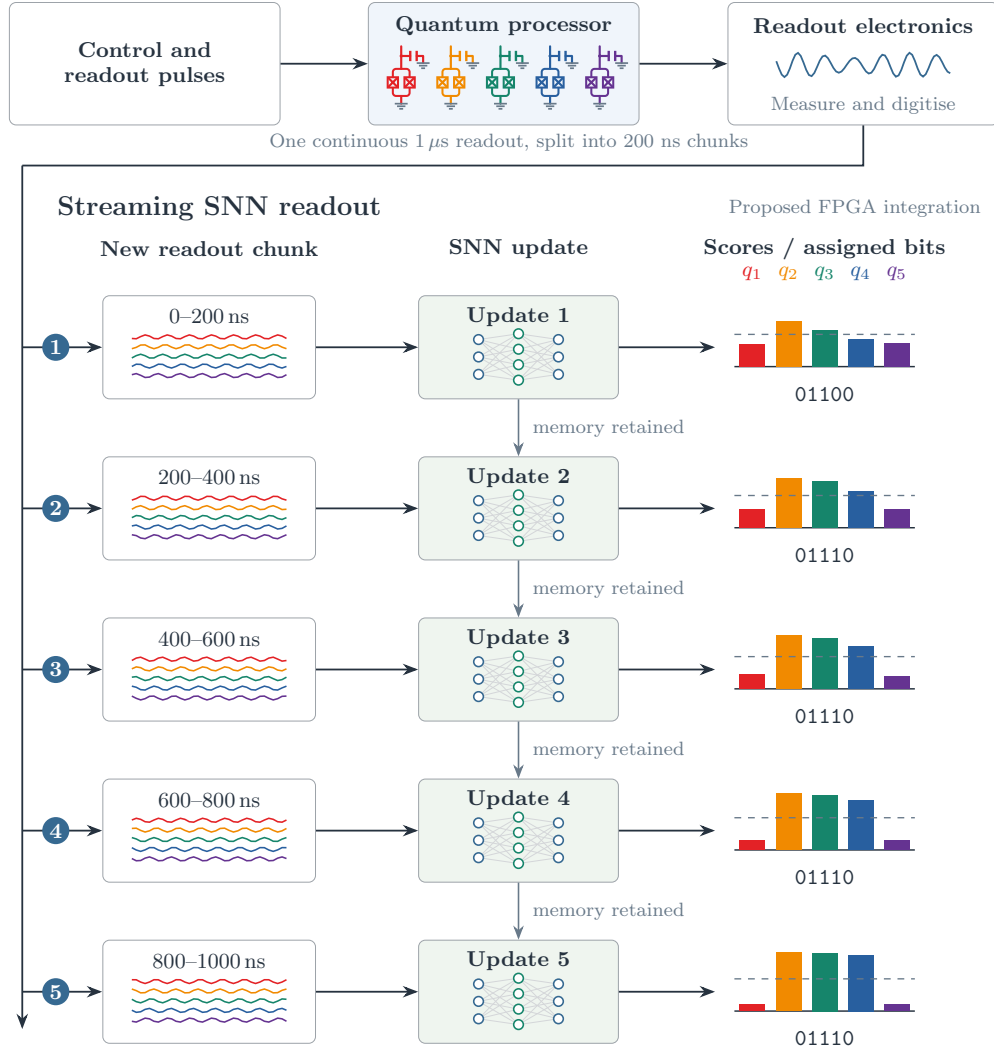


Figure 1: This figure depicts the real-time updates of the SNN measuring qubits in the 01110 state over 200ns timesteps. The coloured lines represent the demodulated traces for each qubit. These are then averaged and fed directly to the SNN, which updates its internal neuron states and predictions for the qubit readout classes.

ture (IQ) traces. Thus, each sample contains the temporal IQ response of the frequency-multiplexed readout chain, rather than only an integrated scalar per qubit. This makes the dataset reusable for benchmarking discriminators that exploit either conventional integrated features, full-time-domain trajectories, correlations among readout channels, or hardware-efficient compressed representations.

The five transmons, individual dispersive readout resonators, shared frequency-multiplexed feedline, labelled preparation of all 32 computational-basis states, and time-resolved IQ acquisition provide the experimental setting used throughout this work. The following section describes how these traces are processed and introduces the benchmark methods and performance metrics.

2.1 Qubit-State Readout Traces

We restrict the analysis of the five-qubit frequency-multiplexed readout traces to the first $1\text{ }\mu\text{s}$ of each acquisition. The sampling period is $\Delta t = 2\text{ ns}$, yielding $N = 500$ samples per trace. The in-phase (I) and quadrature (Q) samples oscillate at their respective intermediate frequency (IF) and are combined as

$$z[n] = I_{\text{IF}}[n] + iQ_{\text{IF}}[n], \quad n = 0, \dots, N-1. \quad (1)$$

For qubit q , the trace is demodulated at IF, f_q ,

$$x_q[n] = z[n] \exp(i2\pi f_q n \Delta t) = I_q[n] + iQ_q[n]. \quad (2)$$

After demodulation, each qubit trace is smoothed with a centred moving-average boxcar,

$$\tilde{x}_{q,W}[n] = \frac{1}{W} \sum_{r=-\lfloor (W-1)/2 \rfloor}^{\lceil (W-1)/2 \rceil} x_q[\text{clip}(n+r, 0, N-1)]. \quad (3)$$

For the figures showing example traces, we use $W = 5$, while for the matched-filter benchmark, we use $W = 25$. Fig. A.1 shows a representative digitally-unmodulated trace and the corresponding demodulated trace, where we have used the $W = 5$ boxcar on the latter. Fig. A.2 represents the average demodulated response for the prepared ground and excited states, which also uses the $W = 5$ boxcar. The graph reveals that, in all cases, there are clear separations between the ground and excited states in at least one channel, except for q_2 , where the separation is more subtle.

The performance metric used in this work is state-assignment fidelity, i.e. the probability that the predicted assignment is correct. For qubit q , the single-qubit assignment fidelity is

$$F_q = \frac{1}{2} \left[P(\hat{b}_q = 0 \mid b_q = 0) + P(\hat{b}_q = 1 \mid b_q = 1) \right], \quad (4)$$

where b_q is the prepared bit and $\hat{b}_q$ is the assigned bit. It represents the quality of the qubit readout, where a value of 1 would imply a perfect classification of the qubit state. Since we consider multiqubit readout traces, we focus most of our attention on the geometric mean of the five scores,

$$F_{\text{geom}} = \left(\prod_{q=1}^5 F_q \right)^{1/5}. \quad (5)$$

Another possibility is to use the accuracy, defined as the fraction of samples for which all five bits are assigned correctly. However, this is much more sensitive to outliers, since an assignment that is incorrect by one bit flip is considered just as incorrect as one that is incorrect in all five bits. To diagnose readout crosstalk, we use the cross-fidelity matrix

$$F_{ij}^{\text{CF}} = 1 - P(\hat{b}_i = 1 \mid b_j = 0) - P(\hat{b}_i = 0 \mid b_j = 1), \quad (6)$$

where i indexes the assigned qubit and j indexes the prepared qubit. The diagonal gives the single-qubit discrimination contrast, $F_{ii}^{\text{CF}} = 2F_i - 1$ for balanced test labels, while off-diagonal entries quantify how the prepared state of one qubit biases the assignment of another.

2.2 Matched Filters

The matched filter (MF) is a good baseline method for this qubit discrimination task. It uses the demodulated and boxcar-smoothed I, Q trace for each qubit. For each qubit, the ground- and excited-state template means $\mu_{q,0}$ and $\mu_{q,1}$, and variances $\sigma_{q,0}^2$ and $\sigma_{q,1}^2$, are estimated from the training split. For the MF estimation, we only use traces with the other four qubits prepared in the ground state¹. The filters are vectors with dimensions equal to the number of timesteps in the trace, and there is a filter for each (I,Q) trace of each qubit. The Fisher-style kernel defines the filters as

$$h_q = \frac{\mu_{q,1} - \mu_{q,0}}{\sigma_{q,1}^2 + \sigma_{q,0}^2 + 0.05 m_q}, \quad h_q \leftarrow \frac{h_q}{\|h_q\|_2}, \quad (7)$$

where m_q is the median of $\sigma_{q,1}^2 + \sigma_{q,0}^2$. For a test trace j , the MF score is the dot product between the filter and the corresponding demodulated trace

$$s_{j,q} = \langle h_q, \tilde{x}_{j,q} \rangle. \quad (8)$$

One threshold per qubit is chosen on the training split,

$$\theta_q^* = \arg \max_{\theta} \frac{1}{2} [P(s_{j,q} \leq \theta \mid b_q = 0) + P(s_{j,q} > \theta \mid b_q = 1)], \quad \hat{b}_{j,q} = \mathbb{I}[s_{j,q} > \theta_q^*]. \quad (9)$$

The MF benchmark reaches $F_{\text{geom}} = 0.8960$, with most of the loss coming from the second qubit.

2.3 Readout Errors

The main sources of readout error are noise in the measurement chain, qubit-state transitions during the acquisition, and crosstalk between simultaneously measured channels. For a qubit prepared in the excited state, energy relaxation can occur before or during the readout pulse. The resulting trace initially follows the excited-state response before moving towards the ground-state response. An MF that integrates the complete trace into a single score can therefore place these events between the two main state distributions or incorrectly

¹Although in the end, this had a very small effect on the results compared to allowing the other qubits to be in either state.

assign them to the ground state. Time-resolved classifiers can instead identify the transient trajectory associated with relaxation [8, 17].

Frequency-multiplexed readout introduces additional errors because several readout tones, resonators, and qubits share the same feedline and amplification chain. Crosstalk can arise from insufficient spectral separation between resonators, parasitic coupling between qubits and neighbouring readout resonators, residual photon populations, and interactions between signals in the amplifiers, mixers, and digitisation electronics [8]. These effects cause the measured response of one qubit to depend on the states of the other qubits, so the joint readout distribution is not generally separable into five independent classification problems.

Other nonidealities include imperfect state preparation, thermal excitation, amplifier noise, and measurement-induced state transitions. Together, these effects produce broadened, distorted, and sometimes non-Gaussian distributions in the integrated (I,Q) plane. A multiqubit classifier can learn correlations between channels and compensate for some of these system-dependent effects. At the same time, access to the time evolution of the traces provides information that is lost by complete-window integration.

2.4 Machine-Learning Approaches

Conventional superconducting-qubit readout pipelines typically integrate the IQ trace using boxcar or matched-filter weights and then classify each qubit independently using thresholds or Gaussian-style discriminators. These approaches are well matched to isolated single-qubit readout, but they become suboptimal for simultaneous multiqubit measurement because the joint measurement distribution is not generally separable across channels. In particular, multiplexed readout can produce correlated assignment errors and state-dependent distortions that are not captured by independent single-qubit decision boundaries [8].

ML methods provide a natural extension of matched filtering because they can learn nonlinear decision boundaries directly from calibrated readout data. Early work showed that support-vector machines and linear discriminant analysis can improve single-shot state assignment beyond simple thresholding [18]. Subsequent approaches employed time-resolved models, such as recurrent neural networks [19], hidden Markov models [17], and reservoir computing methods [11, 20], to exploit the temporal structure of the measurement record. For multiplexed superconducting readout, neural-network classifiers have also been shown to improve multiqubit assignment by learning correlations between channels and reducing state-dependent crosstalk [8, 21]. Other recent work has combined learned models with physically motivated readout features, Bayesian inference, calibration routines, and FPGA deployment workflows [10, 12, 22, 23].

Lienhard et al. used fully connected feed-forward neural networks as multiqubit state discriminators operating on the multiplexed readout record. Trained and evaluated on the same five-qubit dataset used here, these networks improved multiqubit state assignment relative to contemporary conventional discriminators, achieving up to a 25% reduction in assignment error and up to an order-of-magnitude reduction in readout crosstalk [8].

The main advantage of these methods is that they use more information than a single integrated score. In particular, access to the time evolution of the trace can retain information about transient effects such as early relaxation. At the same time, multiqubit inputs allow the classifier to correct correlated readout errors. Our SNN approach retains these

advantages while processing the readout stream sequentially, allowing the assignment to be updated in real time and enabling early classification of qubit states.

3 Spiking Neural Networks

SNNs replace the continuous stateless activations of ANNs with spike-events and have an internal dynamical state that persists and changes in time [14,15], and so are often described as the third generation of neural networks. They deviate from the traditional view of the ANNs and provide a much closer model of how the brain processes information. Deep learning with SNNs has also proven successful [24,25]. It has generated a lot of interest in processing temporal data on hardware that achieves low power and latency for sparse, event-like spiking data. The development of dedicated neuromorphic processors such as TrueNorth [26], Loihi [27,28], and SpiNNaker [29] has shown that the scalability of large-scale event-based computing systems is a realistic goal. In scientific domains, many instruments and experimental setups already use synchronous FPGAs for low-latency, real-time machine learning. SNN designs for deployment on FPGAs are digital, clock-driven systems that preserve stateful neuron dynamics and temporal readout and, in some cases, achieve lower power consumption due to sparse spiking activity.

In recent years, interest in SNNs for low-latency scientific applications has grown [30]. Applications include temporal scientific data classification [31], radiation detection [32], and vertex reconstruction in neutrino-nucleus interactions [33]. At the Large Hadron Collider (LHC), SNNs have been studied for applications such as data filtering [34], particle tracking [35], readout for hadron calorimeters [36], and real-time anomaly detection [37]. In this work, we use the temporal state within the SNN to enable the real-time processing and updating of the qubit readout decision.

SNNs are constructed from spiking neurons. The most widely used spiking neuron is the Leaky Integrate and Fire (LIF) neuron, described by

$$y_t = \begin{cases} 0 & \text{if } u_t < u_{\text{thresh}} \\ 1 & \text{if } u_t \geq u_{\text{thresh}} \end{cases} \quad (10)$$

$$u_{t+1} = \beta u_t + w x_{t+1} - \beta y_t u_{\text{thresh}} + b. \quad (11)$$

This describes a neuron receiving an input current x_t (at timestep t) contributing to its membrane potential u_t , i.e. the internal state of the spiking neuron. Over time steps, spike events arrive and accumulate at the neuron. The potential u_t accumulates these current contributions while also leaking potential over time, with $0 \leq \beta \leq 1$ controlling the leakage rate. Once the membrane potential u_t exceeds the neuron’s threshold potential u_{thresh} , the neuron generates a spike and the membrane potential resets. The spiking action is not differentiable, but this nonlinearity gives SNNs their expressivity.

The spiking neuron is used in the same way as a ReLU neuron in an ANN, with layers of neurons separated by linear transformations parameterised by weights and biases. To optimise the weights, biases, and parameters of the spiking neurons, we use surrogate-gradient methods. A differentiable proxy for the spike derivative is used during backpropagation, while on the forward pass, the non-differentiable spiking dynamics are kept [25,38,39]. We

use the `snnTorch` package for training the SNNs in this paper [40]. The `snnTorch` package exposes spiking neurons as `PyTorch` [41] modules and provides tools for training SNNs with modern deep-learning workflows.

3.1 Hardware Acceleration with `hls4ml`

A key part of this work is demonstrating that SNNs can perform the readout within the strict resource and latency requirements dictated by both the μs timescales on which quantum controllers operate and the timesteps we choose for our SNN readout. For this, we use the `hls4ml` package [42–44], which is designed to map trained neural networks to HLS firmware for low-latency FPGA and ASIC applications. Recent work has extended the functionality to support SNNs [16], which we use to translate trained SNN models into HLS code and obtain C-synthesis estimates with Vitis HLS. For specific details on how SNNs are synthesised, including details on window-size logic, we refer the reader to [16].

4 SNN-based Qubit Readout

The sampling period for the readout traces is 2ns, which is much too short to have the SNN read each update individually. As the data streams in, an analogue-to-digital converter (ADC) will split the stream into T chunks and compute the average for each chunk before passing the data chunk by chunk to the SNN. We consider two options $T \in \{5, 10\}$ for the number of chunks, corresponding to chunk durations of 200ns and 100ns, respectively. For timestep t , define $M = N/T$ and write the chunked data stream as

$$\mathcal{S}_t = \{(t-1)M, \dots, tM-1\}. \quad (12)$$

We then define the local feature vector as just the average of the (I, Q) chunk for each qubit, forming a 10-dim vector

$$L_5(t) = \left[\frac{1}{M} \sum_{n \in \mathcal{S}_t} I_{q,5}[n], \quad \frac{1}{M} \sum_{n \in \mathcal{S}_t} Q_{q,5}[n] \right]_{q=1}^5, \quad (13)$$

where $I_{q,5}$ and $Q_{q,5}$ are the real and imaginary parts of $\tilde{x}_{q,5}$. We also define a cumulative feature vector which sums all previous contributions to the traces,

$$C_5(t) = \left[\frac{1}{tM} \sum_{n=0}^{tM-1} I_{q,5}[n], \quad \frac{1}{tM} \sum_{n=0}^{tM-1} Q_{q,5}[n] \right]_{q=1}^5. \quad (14)$$

The L_5 and C_5 features are visualised in Fig. 2, using the I and Q components of q_5 to compare the 5-step and 10-step features. The 5 vs 10 timestep comparison shows that the additional timesteps resolve features that could be important for classifying the qubit state, particularly in the I traces of the L_5 feature.

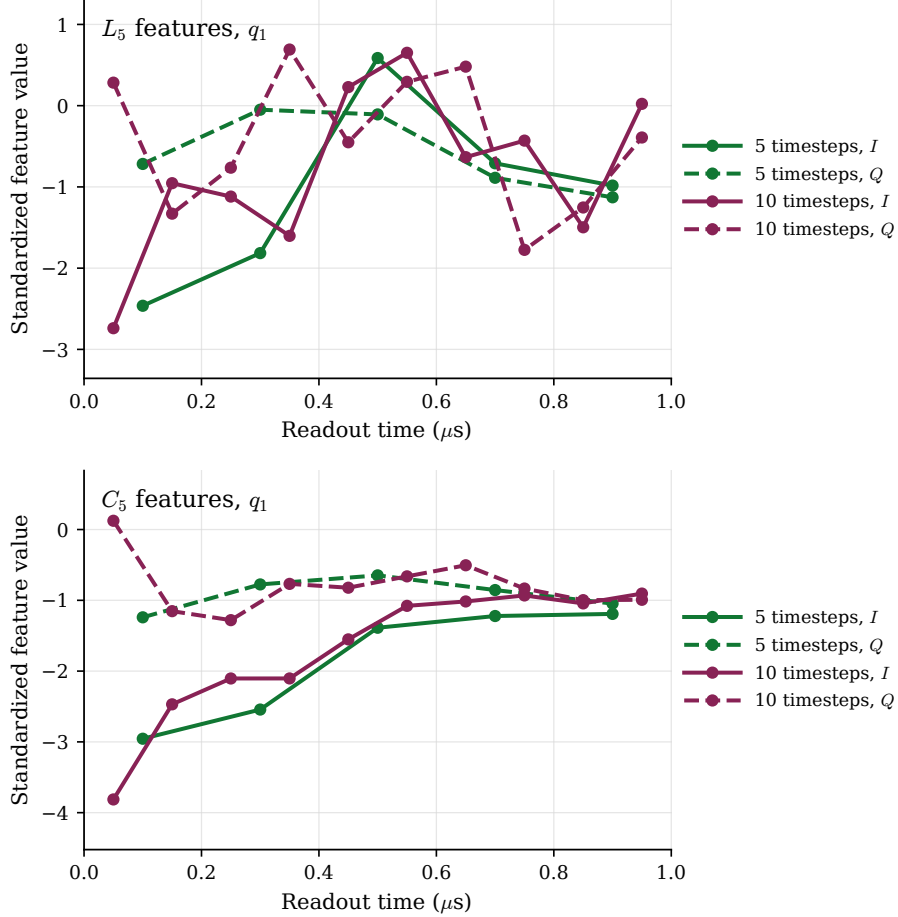


Figure 2: Local L_5 feature values (top) and cumulative C_5 feature values (bottom) for q_5 , comparing $T=5$ and $T=10$ readout summaries over the $1\mu\text{s}$ acquisition window. Solid lines show the I component and dashed lines show the Q component.

4.1 Network Architecture & Training

We use either just the local L_5 features or the combination of the L_5 and cumulative C_5 features as inputs to the SNNs. The features are standardised using the training split mean and standard deviation. The data is split into 432000 training traces and 48000 validation traces, with a separate 1120000-trace test set used only for final evaluation. Each split has an equal number of traces from each of the 32 five-qubit states. All models are trained to predict the five readout bits. The class labels $y_i \in \{0, \dots, 31\}$ are converted to a bit vector $\mathbf{b}_i \in \{0, 1\}^5$ for compatibility with the 5-dim output layer used in the networks.

Each SNN has either a 10-dim or a 20-dim input layer. The input layer is followed by a leaky integrate-and-fire (LIF) hidden layer with either 32, 64, or 128 neurons and a dense 5-dim output layer. We also have an ANN baseline that uses a single hidden layer with 128 neurons over the flattened time-series features. The input dimension for the ANN is either 10 for the L_5 features or 20 for the $L_5 + C_5$ features, multiplied by the number of chunks, either 5 or 10. The input dimension for the ANN, therefore, ranges from 50 to 200.

The ANN baseline flattens the full sequence $\mathbf{x}_i \in \mathbb{R}^{T \times d}$ and so the architecture can be

summarised as

$$\mathbb{R}^{Td} \xrightarrow{\text{Linear}} 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{Dropout}(0.02)} 128 \xrightarrow{\text{Linear}} 5.$$

The SNN processes one timestep at a time using a dense layer, one leaky integrate-and-fire hidden layer, dropout, and a dense five-bit readout, and so can be summarised as

$$\mathbb{R}^d \xrightarrow{\text{Linear}} h \xrightarrow{\text{LIF}} h \xrightarrow{\text{Dropout}(0.02)} h \xrightarrow{\text{Linear}} 5, \quad h \in \{32, 64, 128\}.$$

The LIF hidden neurons use a learnable β parameter (initialised at $\beta = 0.75$), a threshold of $u_{\text{th}} = 1.0$, and a subtractive reset. The readout uses membrane accumulation with no readout decay $\beta_{\text{ro}} = 1.0$, producing five logits $\mathbf{z}_{i,t} \in \mathbb{R}^5$ after each timestep. Final bit predictions are obtained by applying a fixed threshold at zero. Fig. 1 depicts the chunk-by-chunk processing and the corresponding updates to the internal states and qubit assignments.

For a minibatch $\mathcal{B}$, the training loss is bitwise binary cross-entropy with logits. Let

$$\ell(\mathbf{z}_{i,t}, \mathbf{b}_i) = \frac{1}{5} \sum_{q=1}^5 [-b_{iq} \log \sigma(z_{iq,t}) - (1 - b_{iq}) \log (1 - \sigma(z_{iq,t}))].$$

For the default paper models, the SNN loss is applied only after the final chunk,

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(\mathbf{z}_{i,T}, \mathbf{b}_i).$$

The ANN uses the same bitwise binary cross-entropy objective for its single five-logit output.

Models are trained with a minibatch size of 1024, and the best checkpoint is selected based on the validation geometric-mean assignment fidelity. Early stopping is triggered after 30 epochs without an improvement larger than 10^{-5} . The maximum number of epochs allowed is 250, but all models triggered early stopping before this. All reported model results in the paper are obtained from a single training run with a fixed random seed, due to the large number of FP32 and QAT configurations studied and subsequently synthesised with `hls4ml`. However, repeated training runs indicate that the fidelity fluctuates at the level of $\mathcal{O}(10^{-4})$ with different seeds. A summary of the hyperparameters used is given in Tab. 1.

4.2 Results

Tab. 2 compares the MF, ANN, and representative SNN configurations. We see that the 10-step ANN gives the highest score here, with $F_{\text{geom}} = 0.9100$. The best SNN result in this scan is the SNN with a 128-dim hidden layer, the $L_5 + C_5$ feature set, and $T = 10$, which achieves $F_{\text{geom}} = 0.9084$. The 64-dim $L_5 + C_5$ SNN is close, with $F_{\text{geom}} = 0.9083$. Because the ANN (i) has access to the full time-series of data in one forward pass and (ii) has a much larger network size, we expect its performance to be greater than that of the SNNs. The key point here is that the performance of the ANN is not significantly better, which motivates the trade-off for early classification and real-time updates when using SNNs.

We visualise the comparison between different hyperparameter choices for the ANN and SNN (64-dim) in Fig. 3. The cumulative feature C_5 improves both the $T = 5$ and $T = 10$

Hyperparameter	ANN	SNN
Hidden size(s)	128	32, 64, 128
Input features	$L_5, L_5 + C_5$	$L_5, L_5 + C_5$
Timesteps	5, 10	5, 10
Dropout	0.02	0.02
Optimiser	AdamW	AdamW
Learning rate	1.2×10^{-3}	1.0×10^{-2}
Weight decay	1.0×10^{-4}	0
LR scheduler	ReduceLROnPlateau	ReduceLROnPlateau
Scheduler factor	0.5	0.5
Scheduler patience	10 epochs	10 epochs
Minimum learning rate	1.0×10^{-5}	1.0×10^{-5}
Maximum epochs	250	250
Early stopping patience	30 epochs	30 epochs
Selection metric	validation F_{geom}	validation F_{geom}

Table 1: Model and training hyperparameters used in training the ANN baseline and the SNN.

Model	Features	F_{geom}	F_{q1}	F_{q2}	F_{q3}	F_{q4}	F_{q5}
Matched filter	–	0.8962	0.9688	0.7396	0.9007	0.9360	0.9569
ANN, $T=5$, 128-dim hidden	$L_5 + C_5$	0.9087	0.9698	0.7418	0.9393	0.9459	0.9694
ANN, $T=10$, 128-dim hidden	L_5	0.9100	0.9705	0.7435	0.9416	0.9465	0.9704
SNN, $T=5$, 64-dim hidden	$L_5 + C_5$	0.9070	0.9688	0.7404	0.9372	0.9449	0.9665
SNN, $T=10$, 64-dim hidden	$L_5 + C_5$	0.9083	0.9695	0.7421	0.9388	0.9453	0.9683
SNN, $T=10$, 128-dim hidden	$L_5 + C_5$	0.9084	0.9693	0.7424	0.9388	0.9454	0.9685

Table 2: Summary of the readout results for the ANN baseline and the SNN models.

models, with the $T=10$ $L_5 + C_5$ model giving the best SNN result. However, this is not solely due to the information contained in C_5 , as both the ANN and the SNN can reconstruct the cumulative features internally from just the L_5 features. The driving factor here is simply that adding input features increases the model’s parameter count, thereby increasing the network’s capacity to perform its task. This is particularly impactful for the SNN, where we have chosen network hyperparameters to meet the strict latency and resource requirements of the real-time readout.

Fig. 4 compares the cross-fidelity matrices for the ANN baseline and the 32- and 64-dim $T=10$ $L_5 + C_5$ SNNs. All three learned classifiers reduce the off-diagonal structure seen in the MF benchmark. The largest off-diagonal magnitude is 0.0159 for the ANN, 0.0210 for the 32-dim SNN, and 0.0130 for the 64-dim SNN. The SNN with the 64-dim hidden layer, therefore, gives the cleanest pairwise crosstalk matrix among these three representative models. At the same time, the ANN keeps a slightly larger diagonal contrast on most qubits, consistent with its marginally higher F_{geom} . In all cases q_2 remains the dominant limitation with the ML models improving its diagonal cross-fidelity from the MF value $F_{22}^{\text{CF}} = 0.479$ only to about $0.48 - 0.49$, so the gains for the ML models here mostly come from modest improvements

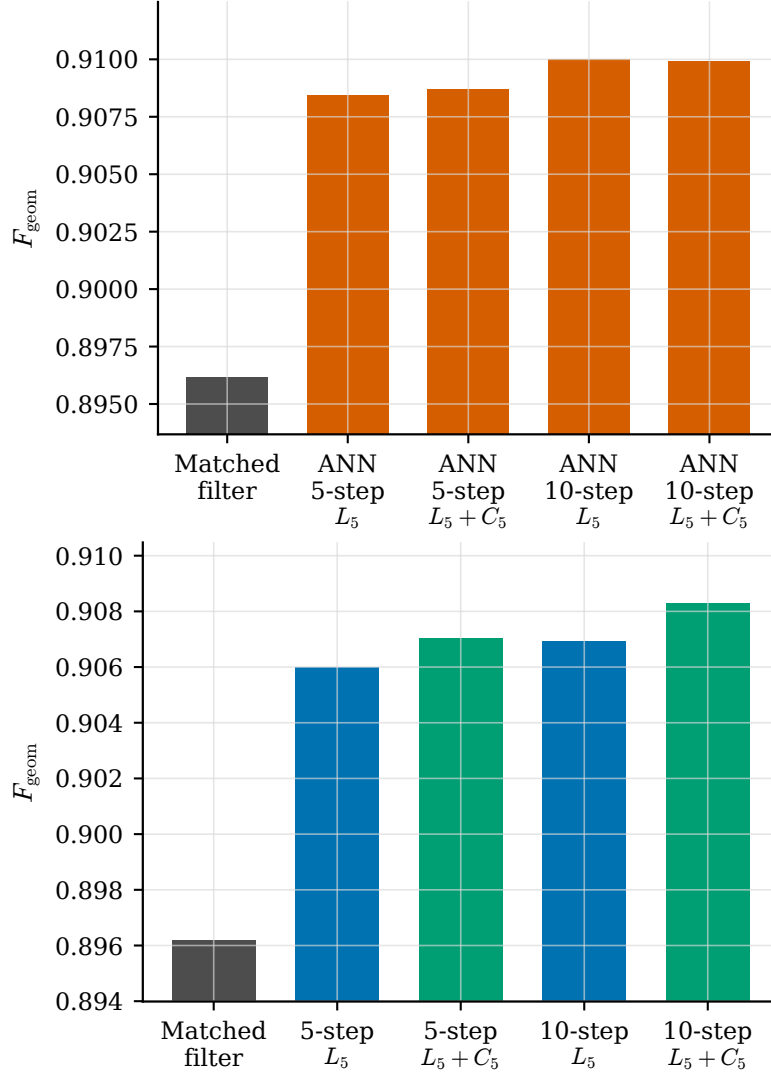


Figure 3: In the plot, we compare ANN models using the geometric mean assignment fidelity. In the plot below, we compare SNN models with a 64-dim hidden layer using the geometric mean assignment fidelity.

across the other qubits.

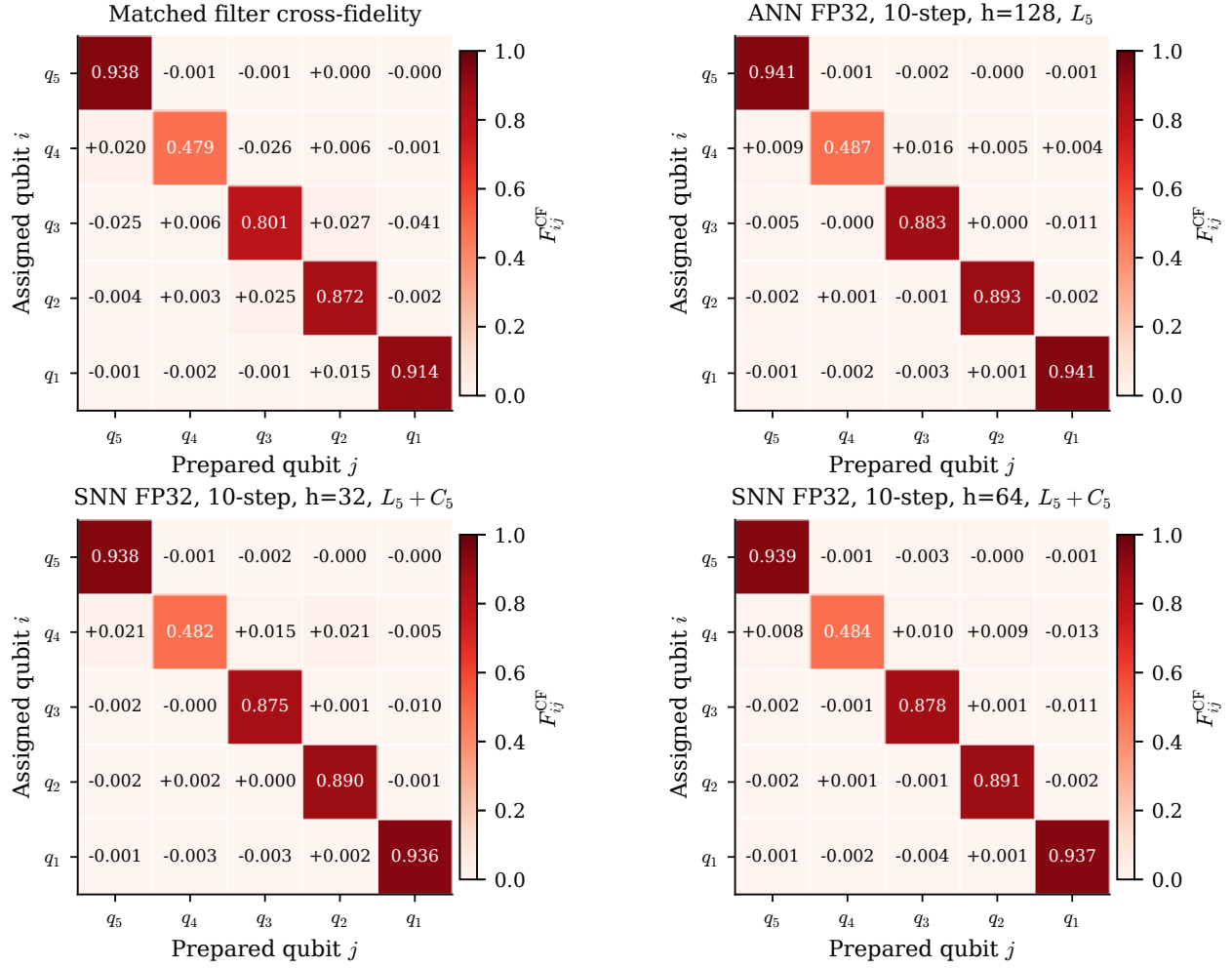


Figure 4: Cross-fidelity matrices for MF baseline (top left), the ANN baseline (top right), the 32-dim SNN (bottom left), and 64-dim SNN (bottom right). All models use $T=10$ and the $L_5 + C_5$ feature set.

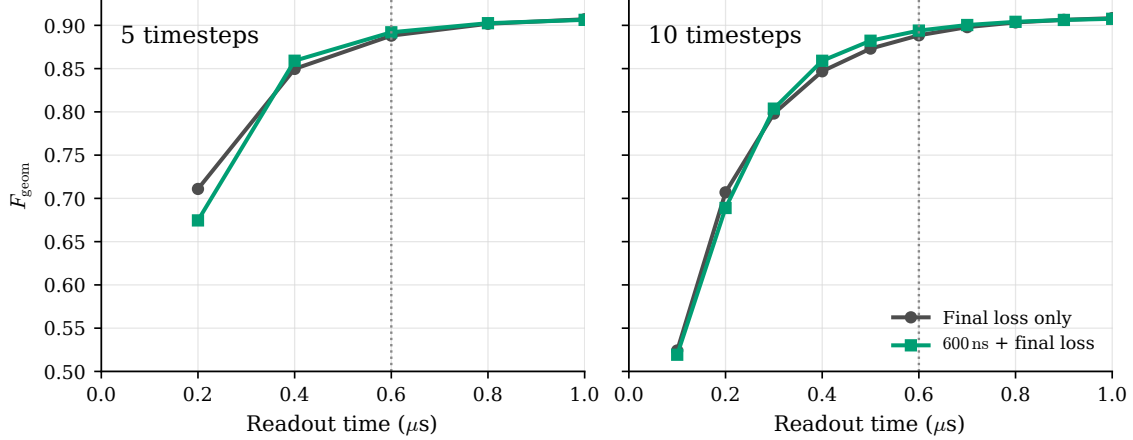


Figure 5: Mid-readout fidelity for the 64-dim SNN trained on $L_5 + C_5$ features. The final-loss-only model is compared with a model trained with equal loss terms at 600 ns and 1 μ s. The dotted vertical line marks the auxiliary-loss timestep.

4.3 Mid-readout optimization

The SNN produces an updated five-bit assignment after each readout chunk, so we can optimise the network on these intermediate results as well as the result at the end of the full 1 μ s window to target high-fidelity results at earlier times. Optimising the SNN at different readout times like this makes it easier to find an optimal trade-off between decision latency and readout fidelity. The baseline models above are trained with a loss on the final timestep only. To test whether the same architecture can be biased toward earlier decisions, we also train the 64-dim SNN on the $L_5 + C_5$ features with an additional binary-cross-entropy term at 600 ns, weighted equally with the final-time loss:

$$\mathcal{L}_{600+\text{final}} = \frac{1}{2|\mathcal{B}|} \sum_{i \in \mathcal{B}} [\ell(\mathbf{z}_{i,t_{600}}, \mathbf{b}_i) + \ell(\mathbf{z}_{i,T}, \mathbf{b}_i)],$$

where $t_{600} = 3$ for the $T=5$ model and $t_{600} = 6$ for the $T=10$ model.

Fig. 5 and Tab. 3 compare the resulting classification fidelity as a function of readout time for the $T=5$ and $T=10$ models. The auxiliary loss does not simply raise the whole curve. It shifts performance toward the middle of the readout window: the earliest chunk is slightly worse, the 300–800 ns region improves, and the final 1 μ s fidelity is essentially unchanged. For the $T=5$ model, the 600 ns fidelity increases from 0.8881 to 0.8920, while the final fidelity changes from 0.9070 to 0.9063. For the $T=10$ model, the 600 ns fidelity increases from 0.8885 to 0.8939, with the largest gain at 400 ns, and the final fidelity changes from 0.9083 to 0.9076. This shows that a small change in the training objective can provide a more useful intermediate classifier without sacrificing the final readout assignment. The value of the early classification is clearer when we look at the per-qubit fidelities produced by the SNN as a function of readout time. Fig. 6 shows that the rate at which the SNN gains confidence in the readout for each qubit is different. We see that for q_5 and q_3 the readout fidelity has plateaued already by 600ns, while q_4 , q_2 , and q_1 improve right up to the end of the window.

Timesteps	Time	Final loss only	600 ns + final loss	Change
5	600 ns	0.8881	0.8920	+0.0039
5	1 μ s	0.9070	0.9063	-0.0007
10	600 ns	0.8885	0.8939	+0.0055
10	1 μ s	0.9083	0.9076	-0.0007

Table 3: Effect of the 600 ns auxiliary loss on the 64-dim $L_5 + C_5$ SNN.

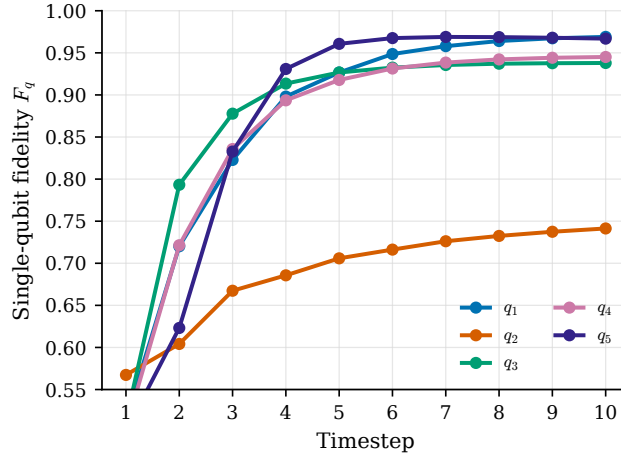


Figure 6: Mid-readout fidelity for each qubit individually with the 64-dim SNN.

5 FPGA Synthesis and Hardware Estimates

The shortest readout chunks considered in this work have a duration of 100 ns. For streaming readout, the central hardware requirement is that each SNN inference update completes within 100 ns, so that the network state and assignment can be updated before the next chunk arrives.

The `hls4ml` package has been used extensively in real-time inference for scientific applications, especially where models need to be designed with resource and latency constraints in mind. The package has been extended to include convolutional layers [45, 46], distributed arithmetic [47], binary/ternary compatibility [48], and SNN inference [16].

The results presented in this section are estimates provided by the AMD Vitis tool for High Level Synthesis from C-language descriptions. Future work will explore the full demodulation, feature construction, data transfer, or FPGA place-and-route stages of a complete readout pipeline.

5.1 Quantisation Aware Training

All results thus far use full 32-bit floating point numbers for all network weights and neuron dynamics. Before synthesising the networks for FPGAs, we fine-tune the training to target the particular fixed-point precision that the networks will use on the FPGA. This is called Quantisation-aware training (QAT). For each SNN, we use W -bit fixed-point weights and neuron potentials with 4 integer bits, where $W \in \{8, 10, 12, 16\}$. We fix the readout mem-

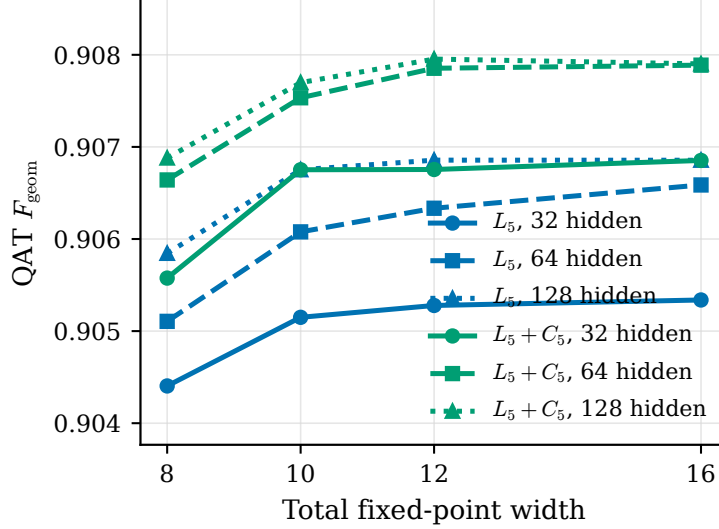


Figure 7: Test fidelity results for the QAT precision sweep with SNNs with both 64- and 32-dim hidden layers, and chunk sizes of 100ns ($T=10$).

brane state to use 16 total bits, of which 8 are integer bits. Throughout the networks, we use rounding rather than truncation to bring numbers to the required precision, and we use saturation rather than wrapping when numbers fall outside the range of the bit precision. The only exception is the accumulator, where we use truncation and wrapping. This is for latency reasons, since adding rounding and saturation to the accumulator can significantly slow down the forward pass on the FPGA. To counter this, we use 18 total bits and 8 integer bits in the accumulator. The QAT routines that we use with the SNNs take these effects into account.

The QAT phase uses the AdamW optimiser with a learning rate 5×10^{-4} , zero weight decay, a maximum of 100 epochs, and early stopping with a patience of 20 epochs. It also uses a ReduceLROnPlateau scheduler with a factor 0.5, a patience of 8 epochs, and a minimum learning rate 10^{-5} . The same bitwise loss and validation-fidelity model selection are used for QAT as for the previous training. The results are shown in Tab. 4 where we see that the 10-step $L_5 + C_5$ model loses only 5×10^{-4} in F_{geom} at $W = 8$, and saturates by $W = 10$. The local-only feature set is consistently lower than $L_5 + C_5$, partly because the larger input feature set yields a larger model, providing more capacity to learn features. The results in all cases approach saturation at $W = 12$.

5.2 Hardware Benchmarks

The hardware benchmarks are obtained using the SNN functionality in `hls4ml`. The synthesis sweep uses the 10-step SNN configurations selected for FPGA synthesis, covering hidden sizes $h \in \{32, 64, 128\}$ and both the L_5 and $L_5 + C_5$ feature sets. As summarised in Tab. 5, all models are generated with `hls4ml` using the Vitis backend, a fully parallel I/O interface, latency-optimised dense layers with reuse factor one, and an 8ns target clock on the XCZU49DR FPGA from the Zynq UltraScale+ family (`xczu49dr-ffvf1760-2-e` device). The weight and activation precision is swept over `ap_fixed<W,4>` for $W \in \{8, 10, 12, 16\}$,

Features	Precision	F_{geom}	F_{q1}	F_{q2}	F_{q3}	F_{q4}	F_{q5}
L_5	<8,4>	0.9051	0.9680	0.7383	0.9309	0.9442	0.9671
L_5	<10,4>	0.9061	0.9683	0.7401	0.9327	0.9447	0.9672
L_5	<12,4>	0.9063	0.9683	0.7411	0.9328	0.9446	0.9672
L_5	<16,4>	0.9066	0.9684	0.7412	0.9337	0.9448	0.9673
$L_5 + C_5$	<8,4>	0.9066	0.9688	0.7405	0.9370	0.9447	0.9646
$L_5 + C_5$	<10,4>	0.9075	0.9693	0.7414	0.9373	0.9449	0.9673
$L_5 + C_5$	<12,4>	0.9079	0.9694	0.7419	0.9378	0.9449	0.9676
$L_5 + C_5$	<16,4>	0.9079	0.9694	0.7420	0.9378	0.9449	0.9677

Table 4: QAT results for the SNN with $T=10$ and a 64-dim hidden layer.

Hyperparameter	Value
Precision sweep	$W4, W \in \{8, 10, 12, 16\}$
Readout precision	<16,8>
Accumulator precision	<18,8>
Backend	Vitis
FPGA part	xczu49dr-ffvf1760-2-e
I/O interface	io_parallel
Target clock period	8.0 ns
Clock uncertainty	18%
Configuration granularity	Layer name
Global reuse factor	1
Global strategy	Latency

Table 5: Hardware hyperparameters used for the FPGA synthesis sweep.

while the readout layer uses `ap_fixed<16,8>` and accumulators use `ap_fixed<18,8>`.

The synthesis results in Tab. 6 show that all configurations achieve an initiation interval of one, with a per-timestep latency between 31 and 52ns. This is below the 100ns duration of the shortest readout chunks considered in this work. The SNN inference can therefore be completed before the next chunk becomes available, allowing the network state and qubit assignments to be updated continuously during the acquisition. The reported latency is for the SNN inference kernel only and does not include demodulation, averaging, or data transfer logic.

As expected, the resource usage increases with the hidden-layer size and with the number of input features. Increasing the hidden size from 32 to 128 approximately scales the number of weights, and consequently the LUT and DSP requirements, by the same factor. The $L_5 + C_5$ models require more resources than the corresponding L_5 models because the input dimension is doubled, although the additional cumulative features improve the readout fidelity. The FF usage remains below 1.3% for all configurations, so the dominant hardware trade-off is between LUT and DSP utilisation.

The choice of numerical precision also changes how the arithmetic is mapped to the device. The 8- and 10-bit configurations use no DSP blocks, with multiplication implemented instead using LUT resources. At 12 and 16 bits, the synthesis tool maps an increasing fraction of the

Features	Hidden	Precision	Clk est. (ns)	II	Lat. cyc.	Lat. (ns)	LUT	LUT (%)	FF	FF (%)	DSP	DSP (%)
L_5	32	$\langle 8, 4 \rangle$	6.4	1	5	32	22,793	5.4	1,736	0.2	0	0.0
L_5	32	$\langle 10, 4 \rangle$	7.7	1	5	39	34,569	8.1	1,960	0.2	0	0.0
L_5	32	$\langle 12, 4 \rangle$	7.7	1	4	31	29,251	6.9	1,846	0.2	247	5.8
L_5	32	$\langle 16, 4 \rangle$	8.0	1	4	32	29,442	6.9	2,181	0.3	441	10.3
L_5	64	$\langle 8, 4 \rangle$	6.5	1	6	39	44,601	10.5	3,142	0.4	0	0.0
L_5	64	$\langle 10, 4 \rangle$	7.7	1	5	39	67,527	15.9	3,822	0.4	0	0.0
L_5	64	$\langle 12, 4 \rangle$	7.7	1	4	31	57,547	13.5	3,599	0.4	502	11.8
L_5	64	$\langle 16, 4 \rangle$	8.0	1	4	32	57,007	13.4	3,962	0.5	896	21.0
L_5	128	$\langle 8, 4 \rangle$	6.6	1	8	52	85,572	20.1	7,462	0.9	0	0.0
L_5	128	$\langle 10, 4 \rangle$	7.7	1	5	39	129,457	30.4	7,391	0.9	0	0.0
L_5	128	$\langle 12, 4 \rangle$	7.7	1	4	31	112,476	26.4	6,857	0.8	1,000	23.4
L_5	128	$\langle 16, 4 \rangle$	8.0	1	5	40	111,756	26.3	9,706	1.1	1,825	42.7
$L_5 + C_5$	32	$\langle 8, 4 \rangle$	6.5	1	5	33	30,675	7.2	2,164	0.3	0	0.0
$L_5 + C_5$	32	$\langle 10, 4 \rangle$	7.7	1	5	39	48,188	11.3	2,651	0.3	0	0.0
$L_5 + C_5$	32	$\langle 12, 4 \rangle$	7.7	1	4	31	38,911	9.1	2,757	0.3	403	9.4
$L_5 + C_5$	32	$\langle 16, 4 \rangle$	8.0	1	5	40	38,414	9.0	3,098	0.4	710	16.6
$L_5 + C_5$	64	$\langle 8, 4 \rangle$	6.5	1	6	39	52,276	12.3	3,952	0.5	0	0.0
$L_5 + C_5$	64	$\langle 10, 4 \rangle$	7.7	1	5	39	93,057	21.9	5,121	0.6	0	0.0
$L_5 + C_5$	64	$\langle 12, 4 \rangle$	7.7	1	4	31	75,760	17.8	5,105	0.6	817	19.1
$L_5 + C_5$	64	$\langle 16, 4 \rangle$	8.0	1	5	40	74,036	17.4	5,635	0.7	1,464	34.3
$L_5 + C_5$	128	$\langle 8, 4 \rangle$	6.5	1	8	52	102,757	24.2	8,977	1.1	0	0.0
$L_5 + C_5$	128	$\langle 10, 4 \rangle$	7.7	1	5	39	182,950	43.0	10,090	1.2	0	0.0
$L_5 + C_5$	128	$\langle 12, 4 \rangle$	7.7	1	4	31	149,219	35.1	9,767	1.1	1,718	40.2
$L_5 + C_5$	128	$\langle 16, 4 \rangle$	8.0	1	5	40	146,346	34.4	10,714	1.3	2,986	69.9

Table 6: Per-timestep Vitis HLS C-synthesis estimates for the SNN hardware sweep on the XCZU49DR target. All runs use the parameters defined in Tab. 5. Latency is reported per HLS top-call/chunk.

arithmetic to DSP blocks, which can reduce the LUT usage and, in several configurations, the number of latency cycles. The 16-bit models marginally exceed the 8ns target clock estimate, reaching 8.026ns, while the lower-precision models meet the target. These C-synthesis estimates demonstrate that the SNN inference kernel can be executed within the interval between successive readout chunks on the target FPGA.

6 Conclusions

In this work, we have proposed and studied Spiking Neural Networks for real-time state assignment from frequency-multiplexed superconducting-qubit readout traces. Unlike classifiers that operate only after the complete readout trace has been collected, the SNN processes successive chunks of the measurement record in real-time as the data arrives and maintains an internal temporal state. This allows the SNN to start the state-assignment process earlier, maximising resource utilisation in a physically constrained setting. The availability of time-dependent readouts also enables the controller to use them before the full measurement window has elapsed.

On the five-qubit dataset, the best full-precision SNN achieves a geometric-mean assign-

ment fidelity of $F_{\text{geom}} = 0.9085$, compared with 0.8960 for the matched-filter benchmark and 0.9100 for the ANN baseline. The SNN therefore recovers most of the performance of the larger ANN while using a compact architecture that processes only one readout chunk at a time. The learned classifiers also reduce the off-diagonal structure in the cross-fidelity matrix. For the representative 64-dim SNN, the largest off-diagonal magnitude is reduced to 0.0130, compared with 0.041 for the MF, showing that joint processing of the multiplexed channels can suppress correlated readout errors. Because the SNN produces an assignment after every timestep, its training objective can be modified to target high fidelity at different times during the readout. This makes it easier to find an optimal trade-off between decision latency and readout fidelity. Adding an auxiliary loss at 600ns increases the 600ns fidelity of the ten-step model from 0.8885 to 0.8939, while changing the final fidelity by only 7×10^{-4} .

QAT preserves the readout performance at the precision required for FPGA synthesis. For the 64-dim $L_5 + C_5$ SNN, an 8-bit implementation reaches $F_{\text{geom}} = 0.9066$, while the performance is effectively saturated by 10-12 bit precision. The FPGA synthesis results give per-timestep latencies of 31 – 52ns, with an initiation interval of one for every configuration considered. These latencies are below the 100ns duration of the shortest input chunks, demonstrating that the SNN can update its internal state and prediction before the next chunk of measurement data arrives. Several configurations require no DSP resources, providing additional flexibility when integrating the discriminator alongside the control and signal-processing logic of a quantum-computing system. The latency and resource usage reported here will improve as the SNN functionality in `hls4ml` is further developed.

The results establish SNNs as a promising approach to low-latency readout of superconducting qubits, combining temporal classification and hardware-efficient inference in a single model. Further improvements could be achieved through confidence-based stopping criteria, timestep-dependent training objectives, and improved choices of spiking neurons. Scaling the method to larger multiplexed systems will also determine how effectively the SNN can exploit temporal and inter-channel correlations as the number of simultaneously measured qubits increases.

Acknowledgment

P.D. and B.L. acknowledge support from the German Federal Ministry of Research, Technology and Space (BMFTR) through the program EQuIPS (Grant No. 13N17232). B.D., J.H., and A.J. are grateful for the use of the computing resources from the Northern Ireland High Performance Computing (NI-HPC) service funded by EPSRC (EP/T022175).

A Readout traces

In Fig. A.1 and Fig. A.2, we show the trace for the first qubit and the average demodulated traces for all five qubits, respectively.

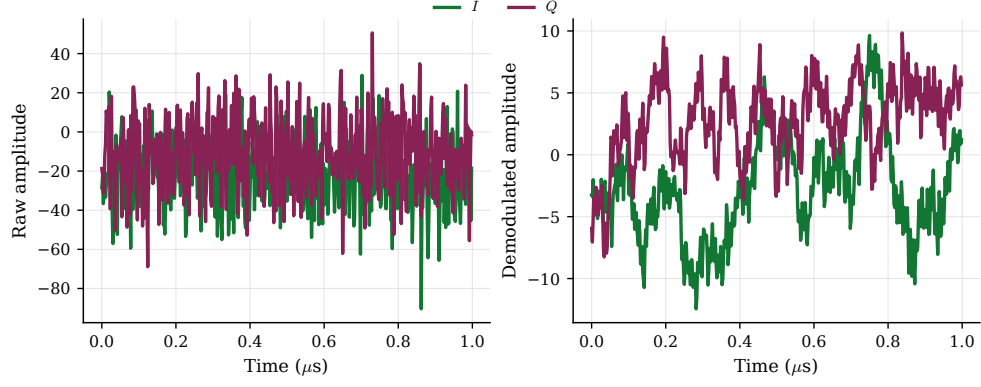


Figure A.1: Representative digitally-unmodulated and demodulated readout trace.

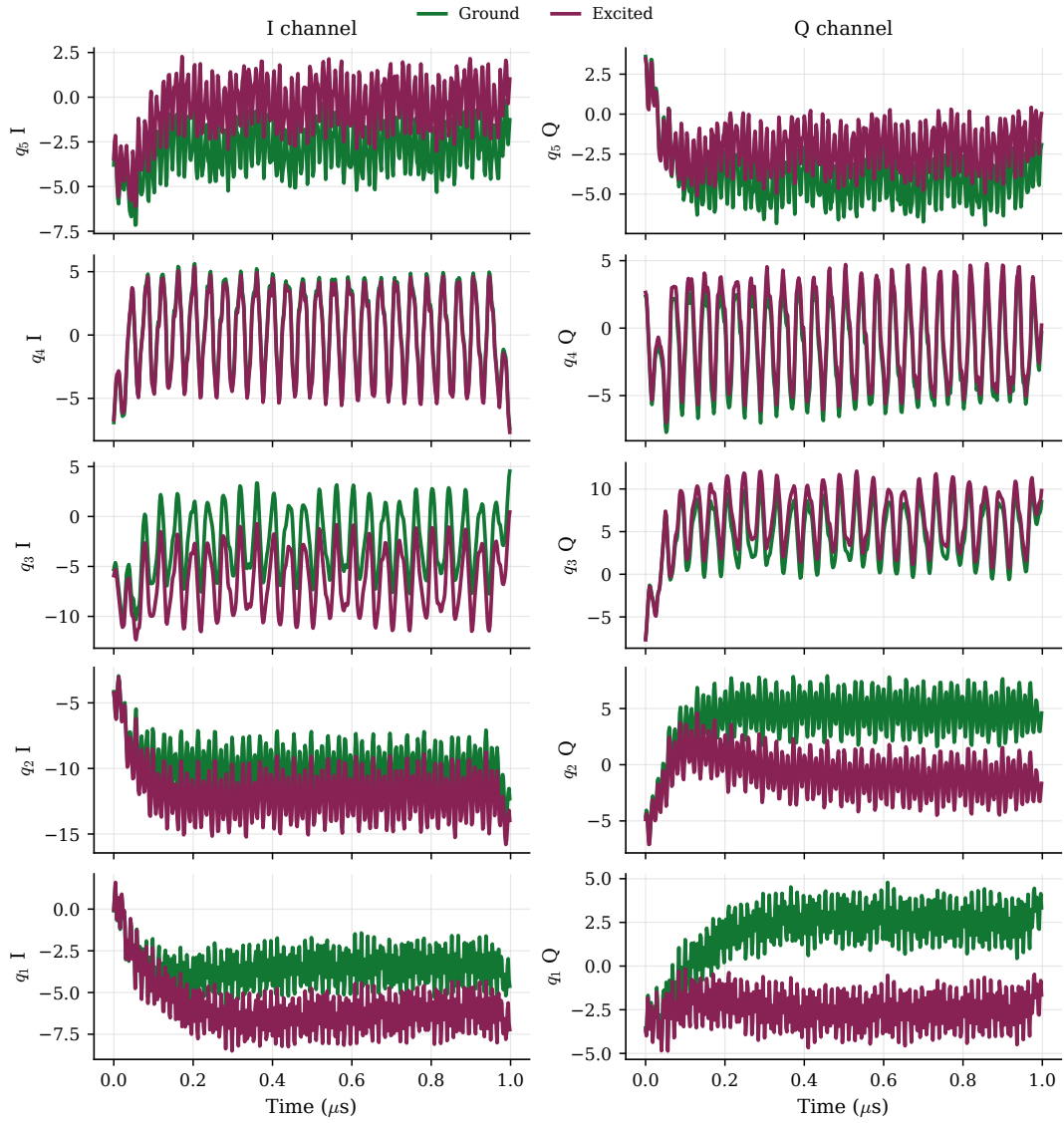


Figure A.2: Average demodulated ground- and excited-state traces for the five qubits.

References

- [1] Ashley Montanaro. Quantum algorithms: an overview. *npj Quantum Information*, 2(1):15023, 2016.
- [2] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Phys. Rev. A*, 52:R2493(R)–R2496(R), Oct 1995.
- [3] A Yu Kitaev. Quantum computations: algorithms and error correction. *Russian Mathematical Surveys*, 52(6):1191, dec 1997.
- [4] Earl T. Campbell, Barbara M. Terhal, and Christophe Vuillot. Roads towards fault-tolerant universal quantum computation. *Nature*, 549(7671):172–179, Sep 2017.
- [5] Julia Cramer, Norbert Kalb, M Adriaan Rol, Bas Hensen, Machiel S Blok, Matthew Markham, Daniel J Twitchen, Ronald Hanson, and Tim H Taminiau. Repeated quantum error correction on a continuously encoded qubit by real-time feedback. *Nature communications*, 7(1):11526, 2016.
- [6] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver. A quantum engineer’s guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, 06 2019.
- [7] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, 638(8052):920–926, Feb 2025.
- [8] Benjamin Lienhard et al. Deep-Neural-Network Discrimination of Multiplexed Superconducting-Qubit States. *Phys. Rev. Applied*, 17(1):014024, 2022.
- [9] Colm A. Ryan, Blake R. Johnson, Jay M. Gambetta, Jerry M. Chow, Marcus P. da Silva, Oliver E. Dial, and Thomas A. Ohki. Tomography via correlation of noisy measurement records. *Phys. Rev. A*, 91:022118, Feb 2015.
- [10] F. Cosco and N. Lo Gullo. Enhancing qubit readout with Bayesian learning. *Phys. Rev. A*, 108(6):L060402, 2023.
- [11] Robert Kent, Benjamin Lienhard, Gregory Lafyatis, and Daniel J. Gauthier. Superconducting-qubit readout using next-generation reservoir computing. *Phys. Rev. Applied*, 25(4):044009, 2026.
- [12] Satvik Maurya, Chaithanya Naik Mude, William D. Oliver, Benjamin Lienhard, and Swamit Tannu. Scaling Qubit Readout with Hardware Efficient Machine Learning Architectures. In *50th Annual International Symposium on Computer Architecture*, 12 2022.
- [13] Kevin Reuer et al. Realizing a deep reinforcement learning agent for real-time quantum feedback. *Nature Commun.*, 14(1):7138, 2023.
- [14] James A. Anderson. A simple neural network generating an interactive memory. *Mathematical Biosciences*, 14(3):197–220, 1972.

- [15] Wolfgang Maass. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, 10(9):1659–1671, 1997.
- [16] Barry M. Dillon. Spiking neural network inference on fpgas with hls4ml, 2026.
- [17] Luis A. Martinez, Yaniv J. Rosen, and Jonathan L. DuBois. Improving Qubit Readout with Hidden Markov Models. *Phys. Rev. A*, 102:062426, 2020.
- [18] Easwar Magesan, Jay M. Gambetta, A. D. Córcoles, and Jerry M. Chow. Machine Learning for Discriminating Quantum Measurement Trajectories and Improving Readout. *Phys. Rev. Lett.*, 114(20):200501, 2015.
- [19] Emmanuel Flurin, Leigh S. Martin, Shay Hacohen-Gourgy, and Irfan Siddiqi. Using a Recurrent Neural Network to Reconstruct Quantum Dynamics of a Superconducting Qubit from Physical Observations. *Phys. Rev. X*, 10(1):011006, 2020.
- [20] Gerasimos Angelatos, Saeed Khan, and Hakan E. Türeci. Reservoir Computing Approach to Quantum State Measurement. *Phys. Rev. X*, 11:041062, 2021.
- [21] Jihye Kim, Byungdu Oh, Yonuk Chong, Euyheon Hwang, and Daniel K. Park. Quantum readout error mitigation via deep learning. *New J. Phys.*, 24(7):073009, 2022.
- [22] Cora N. Barrett et al. Learning-Based Calibration of Flux Crosstalk in Transmon Qubit Arrays. *Phys. Rev. Applied*, 20(2):024070, 2023.
- [23] Giuseppe Di Guglielmo et al. End-to-End Workflow for Machine-Learning-Based Qubit Readout With QICK and hls4ml. *IEEE Trans. Quantum Eng.*, 6:1–10, 2025.
- [24] Amirhossein Tavanaei, Masoud Ghodrati, Saeed Reza Kheradpisheh, Timothée Masquelier, and Anthony Maida. Deep learning in spiking neural networks. *Neural Networks*, 111:47–63, 2019.
- [25] Peter O’Connor and Max Welling. Deep spiking networks, 2016.
- [26] Paul A. Merolla, John V. Arthur, Rodrigo Alvarez-Icaza, Andrew S. Cassidy, Jun Sawada, Filipp Akopyan, Bryan L. Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.
- [27] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.
- [28] Sumit Bam Shrestha, Jonathan Timcheck, Paxon Frady, Leobardo Campos-Macias, and Mike Davies. Efficient video and audio processing with loihi 2, 2023.
- [29] Steve B. Furber, Francesco Galluppi, Steve Temple, and Luis A. Plana. The spinnaker project. *Proceedings of the IEEE*, 102(5):652–665, 2014.

- [30] Adnan Mehonic et al. Roadmap to neuromorphic computing with emerging technologies, 2024.
- [31] Catherine D. Schuman, Thomas E. Potok, Steven Young, Robert Patton, Gabriel Perdue, Gangotree Chakma, Austin Wyer, and Garrett S. Rose. Neuromorphic computing for temporal scientific data classification. In *Proceedings of the Neuromorphic Computing Symposium*, NCS '17, New York, NY, USA, 2017. Association for Computing Machinery.
- [32] S. Lin, S. Ning, H. Zhu, T. Zhou, C. L. Morris, S. Clayton, M. J. Cherukara, R. T. Chen, and Zhehui Wang. Neural network methods for radiation detectors and imaging. *Frontiers in Physics*, Volume 12 - 2024, 2024.
- [33] Linghao Song, Fan Chen, Steven R. Young, Catherine D. Schuman, Gabriel Perdue, and Thomas E. Potok. Deep learning for vertex reconstruction of neutrino-nucleus interaction events with combined energy and time data. In *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 3882–3886, 2019.
- [34] Shruti R. Kulkarni et al. On-sensor data filtering using neuromorphic computing for high energy physics experiments, 2023.
- [35] Emanuele Coradin, Fabio Cufino, Muhammad Awais, Tommaso Dorigo, Enrico Lupi, Eleonora Porcu, Jinu Raj, Fredrik Sandin, and Mia Tosi. Unsupervised particle tracking with neuromorphic computing. *Particles*, 8(2):40, April 2025.
- [36] Enrico Lupi et al. Neuromorphic readout for hadron calorimeters, 2025.
- [37] Barry M. Dillon, Jim Harkin, and Aqib Javed. Anomaly detection with spiking neural networks for LHC physics. *Mach. Learn. Sci. Tech.*, 7(2):025019, 2026.
- [38] Friedemann Zenke and Surya Ganguli. Superspike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 30(6):1514–1541, June 2018.
- [39] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.
- [40] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023.
- [41] Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library, 2019.
- [42] Javier Duarte et al. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation*, 13(07):P07027, 2018.

- [43] Farah Fahim, Benjamin Hawks, Christian Herwig, James Hirschauer, Sergo Jindariani, Nhan Tran, Luca P. Carloni, Giuseppe Di Guglielmo, Philip Harris, Jeffrey Krupa, et al. hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices. In *Proceedings of the tinyML Research Symposium*, 2021.
- [44] Jan-Frederik Schulte et al. hls4ml: A flexible, open-source platform for deep learning acceleration on reconfigurable hardware. *ACM Transactions on Reconfigurable Technology and Systems*, 2026. Just accepted.
- [45] Thea Aarrestad et al. Fast convolutional neural networks on FPGAs with hls4ml. *Machine Learning: Science and Technology*, 2(4):045015, 2021.
- [46] Nicolò Ghielmetti et al. Real-time semantic segmentation on FPGAs for autonomous vehicles with hls4ml. *Mach. Learn. Sci. Tech.*, 2022.
- [47] Chang Sun et al. da4ml: Distributed arithmetic for real-time neural networks on fpgas, 2025.
- [48] Jennifer Ngadiuba et al. Compressing deep neural networks on FPGAs to binary and ternary precision with HLS4ML. *Mach. Learn. Sci. Tech.*, 2:015001, 2021.